

4. System Analysis & Design

1. Problem Statement & Objectives

Problem Statement

Educational institutions lack a unified, automated system to track, analyze, and visualize student academic performance over time. Teachers and administrators manually handle CSV files, spreadsheets, or disparate grade books, making it difficult to identify trends, at-risk students, or attendance patterns. There is a need for a data pipeline that ingests raw student data, cleanses it, stores it in a structured database, and provides visual insights into performance and attendance.

Project Goals

1. Build a Python-based ETL pipeline to collect, clean, and prepare student academic data.
2. Store processed data in a normalized SQL database (SQLite/PostgreSQL).
3. Enable SQL querying for top performers, attendance trends, and monthly score averages.
4. Generate visualizations (score trends, attendance heatmaps) using Matplotlib/Seaborn.
5. Create an interactive dashboard with Streamlit or Plotly Dash.
6. Provide actionable insights for teachers and school administrators.

Use Case Diagram & Descriptions

Actors:

- **Student** (provides data indirectly via CSV)
- **Teacher** (uploads data, views reports, queries performance)
- **System Administrator** (manages database, runs ETL scripts)
- **School Admin** (views aggregated dashboards)

Use Cases:

- Upload CSV Data

- Clean & Preprocess Data
- Import Data into Database
- Generate Top Performers Report
- View Attendance Trends
- View Monthly Score Averages
- Visualize Score Trends
- Visualize Attendance Heatmap
- (Optional) View Interactive Dashboard

Use Case Descriptions (Partial Table)

Use Case	Description	Primary Actor
Upload CSV Data	Teacher provides student scores and attendance CSV file.	Teacher
Clean & Preprocess Data	Python script removes duplicates, fills nulls, categorizes performance.	System
Import into DB	Cleaned data is inserted into normalized SQL tables.	Administrator
Top Performers Report	Query returns students with highest average scores per subject.	Teacher
Attendance Trends	Query shows attendance % over time per student or class.	School Admin
Score Trends	Line chart of average scores by month/subject.	Teacher
Attendance Heatmap	Matrix showing attendance rate per student per day/week.	School Admin

Functional & Non-Functional Requirements

Functional Requirements

ID	Requirement
FR1	System shall load student data from a CSV file with fields: student_id, name, subject, score, date, attendance.
FR2	System shall remove duplicate rows and fill missing scores with subject average.

FR3	System shall convert date strings to datetime objects and categorize scores (High ≥ 80 , Medium 50-79, Low < 50).
FR4	System shall create a normalized relational schema (Students, Subjects, Scores, Attendance).
FR5	System shall support SQL queries for: top performers by subject, attendance trends, average scores by month.
FR6	System shall generate line charts for score trends over time.
FR7	System shall generate heatmaps for attendance data.
FR8	System shall provide an interactive dashboard with filters.

Non-Functional Requirements

ID	Requirement
NFR1	Data cleaning script must execute in < 30 seconds for 10,000 rows.
NFR2	SQL queries must return results in < 2 seconds on a standard laptop.
NFR3	All visualizations must be accessible (colorblind-friendly palettes).
NFR4	System must handle CSV files up to 50MB without crashing.
NFR5	Database schema must be in 3rd Normal Form (3NF).
NFR6	Python code shall follow PEP 8 style guidelines.

2. Database Design & Data Modeling

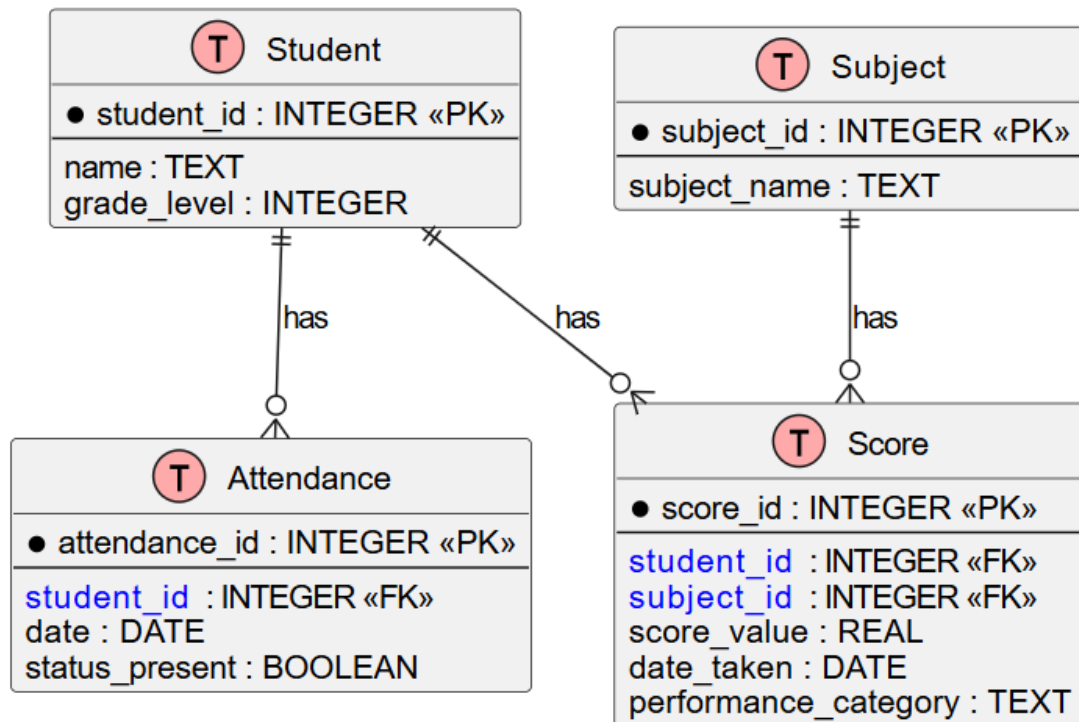
ER Diagram (Entity-Relationship Diagram)

Entities & Attributes:

- **Student** (student_id PK, name, grade_level)
- **Subject** (subject_id PK, subject_name)
- **Score** (score_id PK, student_id FK, subject_id FK, score_value, date_taken, performance_category)
- **Attendance** (attendance_id PK, student_id FK, date, status_present)

Relationships:

- Student —(1) has many (N) Scores
- Student —(1) has many (N) Attendance records
- Subject —(1) has many (N) Scores



Logical & Physical Schema

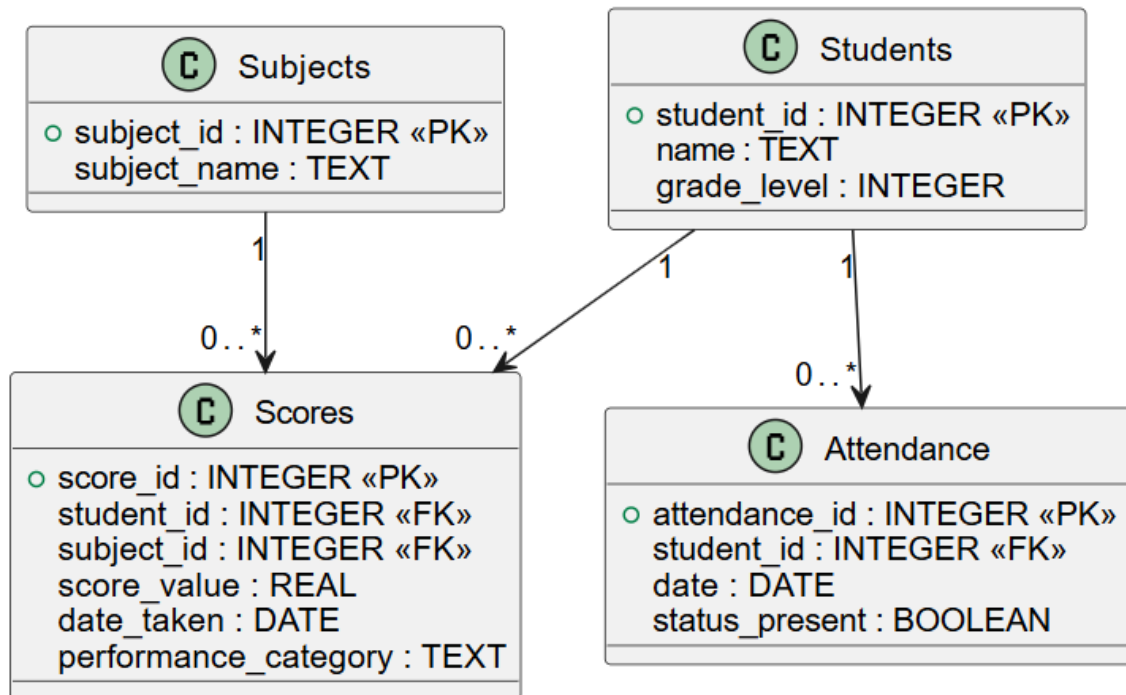
Logical Schema (3NF Normalized)

Students (`student_id`, `name`, `grade_level`)

Subjects (`subject_id`, `subject_name`)

Scores (`score_id`, `student_id`, `subject_id`, `score_value`, `date_taken`, `performance_category`)

Attendance (`attendance_id`, `student_id`, `date`, `status_present`)



Normalization Considerations:

- All tables are in 3NF (no transitive dependencies).
- Performance category is derived but stored for query efficiency (denormalization for reporting).
- Indexes on Scores(student_id, subject_id) and Attendance(date) for query speed.

3. Data Flow & System Behavior

Use Case Diagram

Purpose:

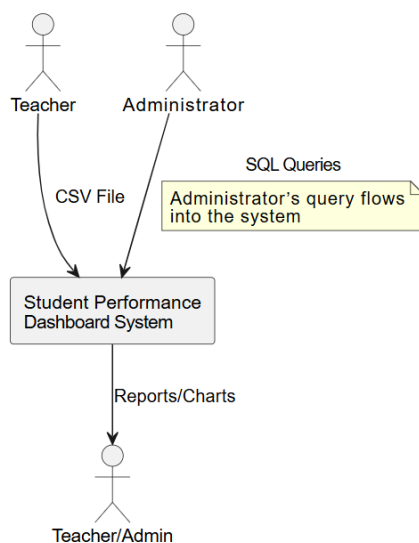
Shows the interactions between external actors (Teacher, Administrator, School Admin) and the system's functionalities (use cases).

Key Elements:

- **Actors:** Teacher, Administrator, School Admin, Student (data source).
- **Use Cases:** Upload CSV, Clean Data, Import to DB, Top Performers Report, Attendance Trends, Score Trends, View Dashboard.
- **Relationships:** Teacher initiates CSV upload; Administrator runs ETL; both generate reports.

Insight:

The diagram clarifies that teachers are the primary data providers, while administrators focus on system operations and report generation. It highlights separation of duties



Sequence Diagram: Generate Top Performers Report

Purpose:

Models the chronological interaction between objects when a teacher requests a “top performers” report.

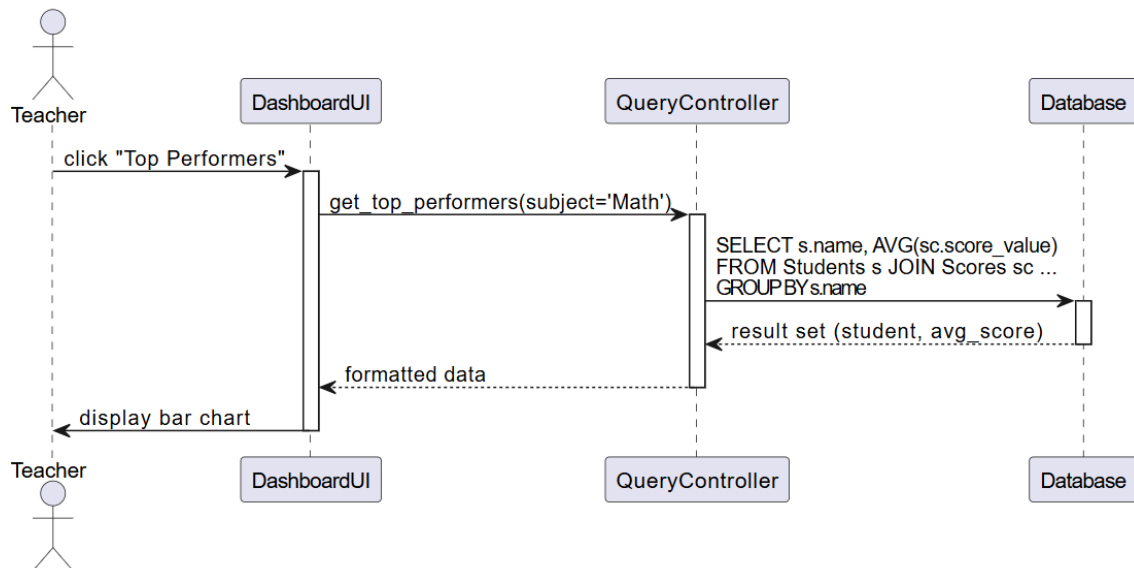
Key Elements:

- **Lifelines:** Teacher, DashboardUI, QueryController, Database.
- **Messages:** click → get_top_performers() → SQL SELECT → result set → formatted data → bar chart.
- **Activation boxes:** Show when each object is processing.

Insight:

The diagram reveals that the DashboardUI delegates business logic to a

QueryController, which isolates SQL generation from the interface, promoting separation of concerns.



Sequence Diagram: Import New CSV Data

Purpose:

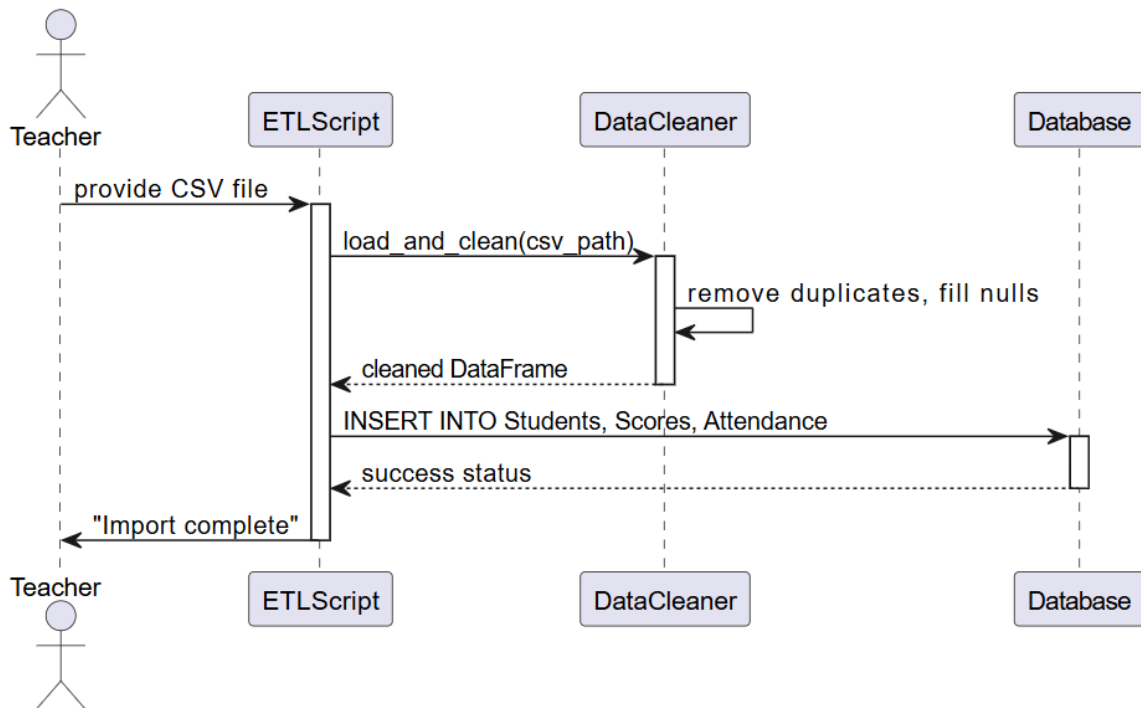
Shows the flow of a complete ETL (Extract, Transform, Load) process from CSV upload to database insertion.

Key Elements:

- **Lifelines:** Teacher, ETLScript, DataCleaner, Database.
- **Self-call:** DataCleaner removes duplicates and fills nulls internally.
- **Return messages:** cleaned DataFrame, success status.

Insight:

The diagram emphasizes that data cleaning is a distinct step before any database operation, ensuring only valid data enters the system. The ETLScript acts as an orchestrator.



Activity Diagram: Teacher Views Score Trends

Purpose:

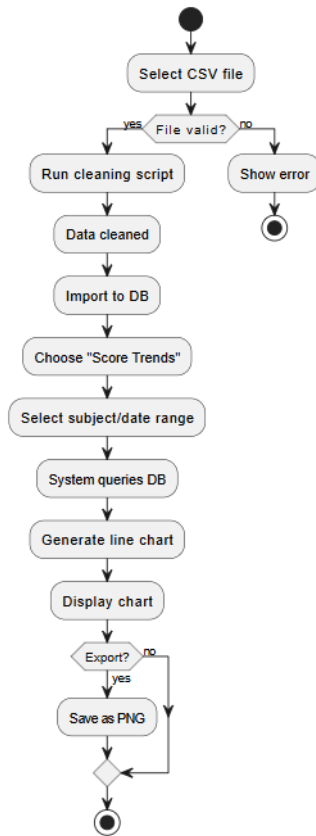
Models the workflow of a teacher generating a score trends visualization, including decision points and alternative paths.

Key Elements:

- **Initial node:** Select CSV file.
- **Decision node:** File valid? (yes/no).
- **Actions:** Run cleaning, import to DB, choose filters, query DB, generate chart, display.
- **Final node:** Save as PNG (optional) or end.

Insight:

The diagram highlights error handling (invalid file shows error and stops). It also includes an optional export step, showing the system supports both on-screen viewing and offline reporting.



State Diagram: Data Processing Job Object

Purpose:

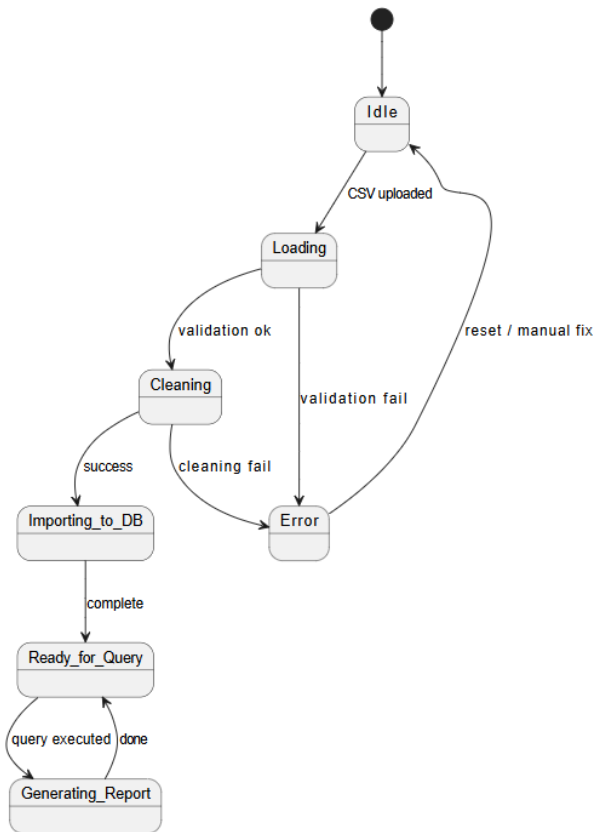
Describes the life cycle of a data processing job from creation to completion or error.

Key Elements:

- **States:** Idle → Loading → Cleaning → Importing to DB → Ready for Query → Generating Report → Error.
- **Transitions:** CSV uploaded, validation ok, success, complete, query executed, done, reset.
- **Alternative paths:** Validation fail or cleaning fail leads to Error; from Error can reset to Idle.

Insight:

The diagram shows that a job can be in “Ready for Query” after a successful import and can generate multiple reports without re-importing. Errors are recoverable via manual reset – useful for debugging



Class Diagram (UML)

Purpose:

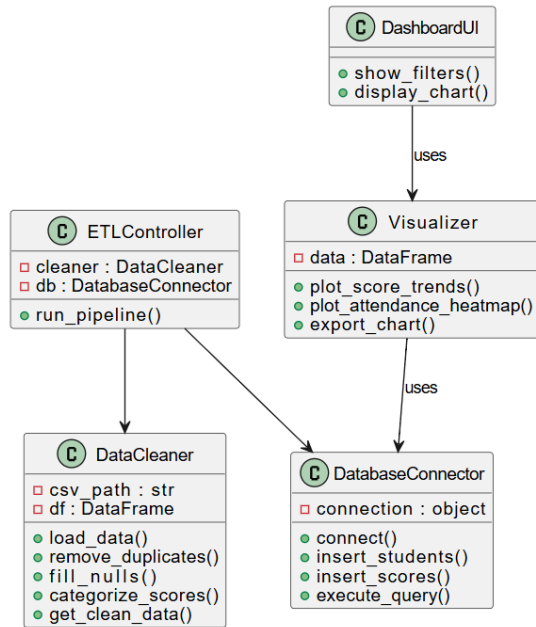
Shows the static structure of the system: classes, attributes, methods, and relationships.

Key Elements:

- **Classes:** DataCleaner, DatabaseConnector, ETLController, Visualizer, DashboardUI.
- **Attributes:** csv_path, df, connection, data.
- **Methods:** load_data(), remove_duplicates(), insert_students(), plot_score_trends(), run_pipeline().
- **Relationships:** ETLController uses DataCleaner and DatabaseConnector (composition/aggregation). Visualizer depends on DatabaseConnector.

Insight:

The diagram reveals a modular design with clear separation: ETLController orchestrates cleaning and DB insertion; Visualizer handles plotting; DatabaseConnector abstracts database operations. This makes the system maintainable and testable.



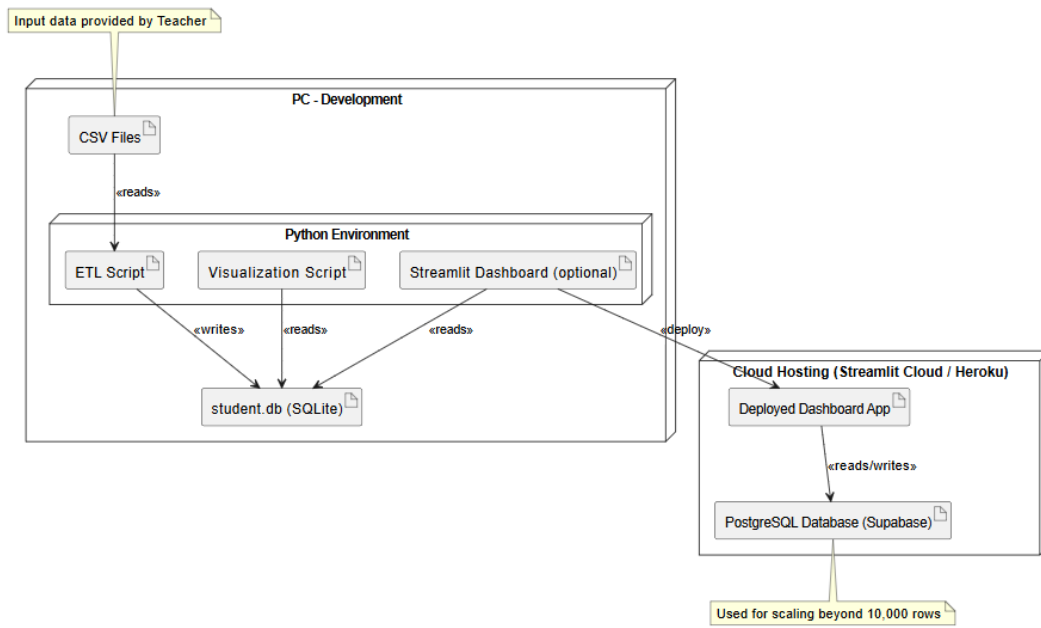
4. System Deployment & Integration

Technology Stack

Tier	Technology Choices
Backend	Python 3.10+
Data Processing	Pandas, NumPy
Database	SQLite (dev), PostgreSQL (prod)
Visualization	Matplotlib, Seaborn, Plotly (optional)
Dashboard (optional)	Streamlit or Plotly Dash
Environment	Jupyter Notebook, VS Code, or CLI
Version Control	Git + GitHub

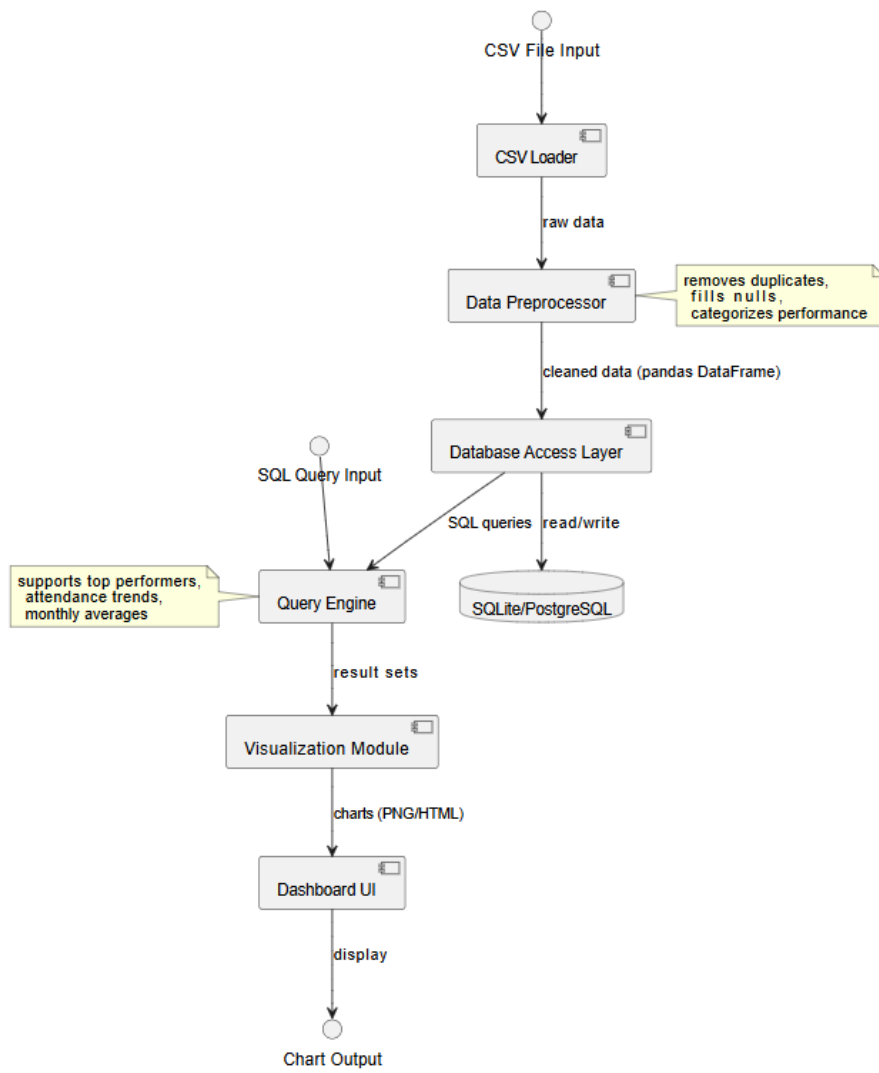
Deployment Diagram

This diagram shows how the software components are distributed across hardware nodes (local development environment and optional cloud deployment).



Component Diagram

This diagram shows the high-level system components and their dependencies.



5. Additional Deliverables

API Documentation (if applicable for optional dashboard)

Endpoint: GET /api/top_performers

Parameter	Type	Required	Description
subject	string	Yes	"Math", "Science", etc.
limit	int	No	Default 5, max 20

Response (JSON):

```
{
  "subject": "Math",
  "top_students": [
    {"name": "Alice", "avg_score": 92.5},
    {"name": "Bob", "avg_score": 88.0}
  ]
}
```

Endpoint: POST /api/upload_csv

- Accepts multipart/form-data with CSV file.
- Returns job_id for status polling.

Testing & Validation

Unit Tests (Python unittest/pytest)

Test ID	Test Case	Expected Result
UT1	test_remove_duplicates()	Duplicate rows removed, count decreases

UT2	test_fill_null_scores()	Null scores replaced with subject avg
UT3	test_categorize_performance(95)	Returns "High"
UT4	test_categorize_performance(45)	Returns "Low"

Integration Tests

Test ID	Scenario	Validation
IT1	CSV → SQLite import	Row count matches after cleaning
IT2	Query "top performers by subject"	Returns non-empty list for valid subject
IT3	Plot generation	PNG file created and non-zero size

User Acceptance Testing (UAT)

Scenario	Success Criteria
Teacher uploads malformed CSV	System shows clear error (not crash)
Admin runs attendance trend query	Result shows % present per week, <2 sec response
User views heatmap	Color legend explains high/low attendance

Deployment Strategy

Hosting Environment (Local/Classroom):

- Python virtual environment (venv)
- SQLite (no external server needed)
- Run scripts via command line or Jupyter

Deployment Pipeline :

Git Push → GitHub Actions →

- Run unit tests
- Build Streamlit app
- Deploy to Streamlit Cloud
- Run smoke test (health check)

Scaling Considerations:

- For >10,000 students: Upgrade to PostgreSQL with indexing.
- For concurrent teachers: Add connection pooling (SQLAlchemy).
- For large CSV files: Implement chunked reading (pandas chunksize).

Rollback Plan:

Keep previous Docker image or Git tag. Database migrations use versioned SQL scripts (e.g., `V1__initial.sql`, `V2__add_index.sql`).